

Reviewer Pack — SVD Participation Dispersion of Mid-Layer Lower-Bounds Read-...

Entry 97b82c049269 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 08:40:36 UTC

SVD Participation Dispersion of Mid-Layer Lower-Bounds Read-Twice BPs for IP₂

Entry ID: 97b82c049269 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 08:40:36 UTC

1. Conjecture

Field A (mathematical branch): Spectral discrepancy / dispersion theory: Lindsey-style $L^{2\text{-vs-}L_\infty}$ transference and the Bourgain-Kashin-Chazelle ‘effective rank’ (participation ratio) of 0/1 incidence matrices, used as a coordinate-free dispersion proxy that is essentially absent from the branching-program literature **Field B** (complexity object): Leveled read-twice branching programs (BPs) computing $IP_2_n = \bigoplus_{i=1}^{n/2} x_i \cdot y_i$; the Borodin-Razborov-Smolensky 1993 read-twice barrier where the read-once vs unrestricted gap is exponential but the read-twice vs unrestricted gap is wide open

Statement:

For a leveled BP P of size s on $N=2n$ variables, let $B_{\text{mid}} \in \{0,1\}^{s \times 2^n}$ be the state-input incidence matrix at the middle level (the level after exactly $\lfloor L(P)/2 \rfloor$ queries on every source-to-sink path), and define the dispersion $DISP(P) := \log_2 ((\sum_i \sigma_i(B_{\text{mid}})^2)^2 / \sum_i \sigma_i(B_{\text{mid}})^4)$; equivalently the log-participation-ratio of the singular spectrum, which always satisfies $DISP(P) \leq \log_2 s$. **CONJECTURE:** every leveled read-twice BP P computing IP_2_n satisfies $DISP(P) \geq n/2 - c \cdot \log_2 n$ for an absolute constant $c \leq 4$, so that $\text{size}(P) \geq 2^{n/2 - c \cdot \log n}$. A single read-twice BP P that correctly computes IP_2_n yet has $DISP(P) < n/2 - 4 \cdot \log_2 n$ falsifies the conjecture.

Rationale (proposer’s reasoning):

Read-twice structure forces every state’s input-fiber to be expressible as the intersection of a ‘first-pass’ subcube with a ‘second-pass’ coset, so the rows of B_{mid} lie in a structured low-discrepancy family — exactly the regime where Chazelle-Bourgain-Kashin participation-ratio dispersion is informative but ordinary rank or σ_2/σ_1 spectral gaps are degenerate (the canonical IP_2 BP has all middle singular values equal, killing naive gap-based bounds). The participation ratio is precisely the right coordinate-free dispersion measure that matches the canonical $2^{n/2}$ read-once IP_2 BP and we conjecture it transfers to read-twice. If true, this gives a ‘distinguishing tensor’ invariant $\rho = DISP$ that is (i) trivially $\leq \log s$ and (ii) $\geq n/2 - O(\log n)$ on IP_2 BPs, yielding $2^{n/2 - O(\log n)}$ read-twice lower bounds — the first matching read-once for read-twice.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): db89fc8b8d1d3e0b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $N \in \{4, 6, 8, 10, 12, 14\}$ with 30 random read-twice BP seeds plus AND/OR/PARITY/-MAJ controls and the canonical read-once IP₂ BP, every BP correctly computing IP₂_N must satisfy $\text{DISP}(B_{\text{mid}}) \geq N/2 - 4 \cdot \log_2 N$ for all $N \geq 8$, AND every control BP (AND/OR/-PARITY/MAJ) must satisfy $\text{DISP} \leq N/4$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL PROOFS	SAFE	0.85	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.92	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - read-twice branching program inner product lower bound - participation ratio singular values communication complexity branching program - spectral dispersion incidence matrix branching program IP₂ lower bound

Top relevant hits considered: - [<http://arxiv.org/abs/1709.08890v2>] Partial matching width and its application to lower bounds for branching programs - [<http://arxiv.org/abs/1007.1875v2>] Lower Bounds for Quantum Oblivious Transfer - [<http://arxiv.org/abs/0906.0693v3>] An improved lower bound on the counterfeit coins problem - [<http://arxiv.org/abs/2210.01601v2>] Quantum communication complexity of linear regression - [<http://arxiv.org/abs/2401.14623v1>] Structure in Communication Complexity and Constant-Cost Complexity Classes - [<http://arxiv.org/abs/1403.8106v1>] Recent advances on the log-rank conjecture in communication complexity - [<http://arxiv.org/abs/1806.02734v3>] Spectral lower bounds for the orthogonal and projective ranks of a graph

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=30.3s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot in column i
        max_row = i
```

```

    for k in range(i+1, n):
        if abs(A[k][i]) > abs(A[max_row][i]):
            max_row = k
    A[i], A[max_row] = A[max_row], A[i]

    # Eliminate entries below pivot
    for k in range(i+1, n):
        factor = A[k][i] / A[i][i]
        for j in range(i, n):
            A[k][j] -= factor * A[i][j]

    # Back-substitute to find solution
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = A[i][-1]
        for j in range(i+1, n):
            x[i] -= A[i][j] * x[j]
        x[i] /= A[i][i]

    return x

def matrix_multiplication(A, B):
    m = len(A)
    n = len(B[0])
    p = len(B)
    C = [[0 for _ in range(n)] for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C

def compute_disp(B_mid):
    s = len(B_mid)
    n = 2 ** (len(B_mid[0]) - 1)
    sigma_squared_sum = sum(sigma**2 for _, sigma in sorted((sum(row), math.sqrt(sum(x*x for x in row)) for row in B_mid)))
    sigma_fourth_sum = sum(sigma**4 for _, sigma in sorted((sum(row), math.sqrt(sum(x*x for x in row)) for row in B_mid)))
    disp = 2 * math.log2((sigma_squared_sum ** 2) / sigma_fourth_sum)
    return disp

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [4, 6, 8, 10, 12, 14]
    results = []

    for N in n_values:
        s = 2 ** (N // 2)
        B_mid = [[random.randint(0, 1) for _ in range(s)] for _ in range(2**N)]

        disp = compute_disp(B_mid)
        results.append({
            "n": N,
            "s": s,
            "disp": disp
        })

```

```

metric_value = sum(result["disp"] for result in results) / len(results)
instances_tested = len(results)
conjecture_holds = all(result["disp"] >= N / 2 - 4 * math.log2(N) for result in results if res
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "DISP",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_disp = sum(r["metric_value"] for r in results) / len(results)
    std_disp = math.sqrt(sum((r["metric_value"] - mean_disp)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_disp} std={std_disp} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

_name': 'DISP', 'metric_value': 17.74664271579066, 'instances_tested': 6, 'conjecture_holds': True,
TRIAL: {'metric_name': 'DISP', 'metric_value': 17.817241456842243, 'instances_tested': 6, 'conjecture_holds': True,
TRIAL: {'metric_name': 'DISP', 'metric_value': 17.772055556368766, 'instances_tested': 6, 'conjecture_holds': True,
TRIAL: {'metric_name': 'DISP', 'metric_value': 17.813045718248656, 'instances_tested': 6, 'conjecture_holds': True,
TRIAL: {'metric_name': 'DISP', 'metric_value': 17.751510874544746, 'instances_tested': 6, 'conjecture_holds': True,
TRIAL: {'metric_name': 'DISP', 'metric_value': 17.858137668491107, 'instances_tested': 6, 'conjecture_holds': True,
TRIAL: {'metric_name': 'DISP', 'metric_value': 17.750584718409037, 'instances_tested': 6, 'conjecture_holds': True,
TRIAL: {'metric_name': 'DISP', 'metric_value': 17.78967489000406, 'instances_tested': 6, 'conjecture_holds': True,
TRIAL: {'metric_name': 'DISP', 'metric_value': 17.805817289705633, 'instances_tested': 6, 'conjecture_holds': True,
TRIAL: {'metric_name': 'DISP', 'metric_value': 17.748623796070323, 'instances_tested': 6, 'conjecture_holds': True,
TRIAL: {'metric_name': 'DISP', 'metric_value': 17.786551890716403, 'instances_tested': 6, 'conjecture_holds': True,
TRIAL: {'metric_name': 'DISP', 'metric_value': 17.793781724798567, 'instances_tested': 6, 'conjecture_holds': True,
TRIAL: {'metric_name': 'DISP', 'metric_value': 17.82067653221687, 'instances_tested': 6, 'conjecture_holds': True,
TRIAL: {'metric_name': 'DISP', 'metric_value': 17.764702392643578, 'instances_tested': 6, 'conjecture_holds': True,
RESULT: SUPPORTED mean=17.79099415384149 std=0.03165067694000282 support_fraction=1.0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The DISP values cluster tightly around 17.79, which is suspiciously close to $\log_2$ of a fixed s (e.g., $s \approx 2^{17.79}$), suggesting the test is measuring a near-uniform spectrum on a single canonical/trivial BP construction rather than searching adversarial read-twice BPs for IP_2 . With only 6 instances and no variation in n shown, we cannot distinguish $DISP \geq n/2 - c \cdot \log n$ from $DISP \approx \log_2 s$ holding vacuously because s itself is large; the bound $DISP \leq \log_2 s$ makes any ‘large s ’ instance trivially

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test reports SUPPORTED with $DISP \approx 17.79$ across all trials, but this near-constant value across only 6 instances suggests metric saturation on a canonical symmetric BP rather than a genuine search over adversarial read-twice BPs; the pre-registered criterion required testing across $N \in \{4, 6, 8, 10, 12, 14\}$ with 30 random seeds plus controls, and the output shows no N -variation or control results, so the support condition is not unambiguously verified. | next: Re-run with explicit per- N breakdowns

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	291602
2	preregistration	claude_max	opus	0	0	8004
3	novelty	claude_max	opus	0	0	3470
4	novelty	claude_max	opus	0	0	8998
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10906
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11654
7	critic	claude_max	opus	0	0	12732
8	judge	claude_max	opus	0	0	6387

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 353754 ms total latency. Provider mix: {'claude_max': 6, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/97b82c049269.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/97b82c049269.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/97b82c049269.tar.gz` (if generated)